

SHORT-VIDEO-ENGINE · @TETOCIENCIA

Autopiloto de producción de videos cortos

Informe de estado: lo que se tiene, lo que se necesita, presupuesto y consejos



Fecha: 1 de octubre de 2026

Repositorio: GerardOtto/Proyecto1 · rama `claude/laughing-pasteur-y6cyjy`

Documentación técnica: `docs/11_AUTOPILOT.md` · `docs/adr/0007-autopiloto.md`

Estado: implementado y probado sin conexión; listo para tu revisión y una primera corrida local con claves reales.

1. Resumen ejecutivo

Se construyó un **autopiloto** (`npm run autopilot`) encima del motor existente. Produce episodios con el **mismo formato** del demo: saludo "¡Papu papu!", rótulo con palabra clave, meme, explicación por turnos, remate y llamada a la acción. En cada episodio varía:

- **personajes y roles** (presentador, contraparte y, a veces, invitado), sin repetir la misma pareja;
- **colores** (cinco temas visuales con fondos animados y gráficos propios);
- **efectos de sonido**, colocados según la reacción, la sección y el personaje;
- **tema**: noticias de IA e informática o, si no hay, conceptos de Computer Science, IA, programación, polémicas e historia.

Qué se verificó en este entorno

- Se planificaron y escribieron 3 episodios distintos (temas, parejas, paletas y formatos), todos válidos para el motor.
- Un episodio (AlphaGo vs Lee Sedol) se produjo de punta a punta: voces de prueba (flite), reajuste a **67,6 s** con el audio real, render en ~5 min y QA con todos los checks *hard* en PASS (licencias en aviso).
- **190 tests en verde**. Entre ellos, uno que genera y valida un guion para cada uno de los 29 temas del banco (duración de 62 a 115 s, rótulo que cabe, turnos correctos). Lint y typecheck limpios.

Lo que no se pudo probar aquí (sin red ni claves)

La lectura real de feeds de noticias, el escritor con Claude (sí se probó con un proveedor simulado), las voces de Fish Audio y el modelo de whisper.cpp. Todo está implementado, pero su primera corrida debe hacerse en tu máquina (sección 6).

2. Cómo funciona

1 • Fuentes

RSS/Atom de medios en español e inglés → puntuación por nicho y frescura → agrupado de la misma noticia en varias fuentes.

2 • Banco evergreen

Si no hay noticias nuevas (o no hay escritor LLM): 29 temas con datos verificables y fuentes.

3 • Plan

Tema + formato + casting por roles + tema visual. Determinista: no repite tema, pareja ni paleta.

4 • Gráficos

Tarjetas propias (cifra, ideas clave, código, comparación, titular) con la paleta del tema. Licencia propia.

5 • Guion

Escritor de plantilla (gratis, sin conexión) o escritor LLM (noticias), en el formato del motor.

6 • SFX

Reglas por *tags*: shock → golpe dramático, risa → risa, idea → ding, firma de Teto, sin pasarse de 7.

7 • Revisión

Lint editorial + validador del motor. El episodio queda en `needs_review`.

8 • Producción

--produce : voces, transcripción, reajuste al audio, render y QA (pipeline existente).

9 • Publicación

Textos para TikTok, Instagram y YouTube, hashtags, comentario fijado, horario (CDMX) y checklist.

```

npm run autopilot                                # 1 episodio listo para revisar
npm run autopilot -- --batch 3                    # semana tipo (máx. 1 noticia)
npm run autopilot -- --episode <id> --produce       # producir tras revisar
npm run autopilot -- --check-feeds                # probar las fuentes de noticias
npm run autopilot -- --make-backgrounds           # fondos de los temas visuales
npm run avatars:ingest -- --character rin --from <carpeta de renders>

```

3. Lo que se tiene

Componente	Qué hace	Estado
Motor de video (previo)	Guion etiquetado → timeline validado → voces → reajuste → Remotion → MP4 1080x1920 + SRT + reporte QA. Rótulo de gancho, b-roll, marca de agua, música opcional.	En producción
Noticias (news.ts)	Lector RSS/Atom, palabras clave del nicho con peso, bloqueo de ofertas o temas sensibles, frescura (vida media de 24 h), agrupado y deduplicación contra el historial.	Probado con fixtures
Banco evergreen	29 temas: 4 conceptos de CS, 7 de IA (incluido "¿Miku es IA?"), 6 de programación, 7 polémicas (left-pad, Heartbleed, Log4Shell, CrowdStrike...) y 5 de historia. Cada uno con 5 puntos, preguntas, remate, idea final, visual y fuentes.	Listo
Planificador	Formatos: noticia explicada, concepto en 80 s, historia de una polémica y mito o realidad. Casting por roles, solo con personajes "listos". Temas visuales por categoría.	Listo
Escritor de plantilla	Turnos intercalados; cada pregunta va después del dato con el que se relaciona; remate propio del tema; relleno si queda corto. Gratis y reproducible.	Listo
Escritor LLM	Prompt con el formato, el demo de ejemplo, el catálogo y el casting. Usa solo los hechos del brief y devuelve las afirmaciones a verificar. Se valida y reintenta hasta 3 veces.	Probado con mock
Director de SFX	Elige efectos por tags del catálogo según reacción, sección y personaje; respeta un máximo y una separación; no toca lo marcado a mano.	Listo
Gráficos y fondos	Generador HTML → PNG con 5 tipos de tarjeta y 5 paletas; 4 fondos animados nuevos, generados con ffmpeg (licencia propia).	Listo
Kit de publicación	Descripciones según el estudio de distribución, hashtags (5/5/3), comentario fijado, horario en CDMX y checklist (incluida la etiqueta de IA).	Listo
Ingesta de avatares	Renders → quitar fondo (remove-bg.py) → tamaño acotado → nombres estándar → reacciones y variantes en characters.json . Crea personajes nuevos.	Listo

Historial	projects/_autopilot/history.json : temas, noticias, parejas y paletas usados, y estado de cada episodio.	Listo
-----------	--	-------

Personajes

Personaje	Avatares	Voz (Fish Audio)	Roles en el autopiloto
Teto	Reales (renders MMD) + variantes	Configurada	Presentadora / contraparte
Miku	Reales + variantes	Configurada	Presentadora / contraparte
Luka	Reales + variantes	Configurada	Presentadora / contraparte / invitada
Rin	Placeholder	Sin configurar	Excluida hasta tener avatares y voz
Len	Placeholder	Sin configurar	Excluido hasta tener avatares y voz

4. Lo que se necesita

Imprescindible para producir en serie

#	Necesidad	Detalle
1	Revisión humana de cada guion	10-15 min por episodio: hechos contra las fuentes, tono, chistes y respeto a la comunidad. El autopiloto no publica solo , por diseño.
2	Voz de cada personaje del casting	<code>voice.fish.referenceId + FISH_AUDIO_API_KEY</code> (o tu flujo actual de archivos con <code>--tts files</code>), y el saludo <code>assets/voice/<id>/papu_papu.mp3</code> por personaje.
3	Clave de Claude (<code>ANTHROPIC_API_KEY</code>)	Solo para noticias (y opcionalmente para dar más variedad a los evergreen). Sin clave, el autopiloto usa solo el banco evergreen.
4	Verificar las fuentes de noticias	<code>npm run autopilot -- --check-feeds</code> . Ajusta las URLs y los pesos en <code>config/autopilot/sources.json</code> .
5	Modelo de whisper.cpp	<code>npm run whisper:install</code> . Sin él, los subtítulos usan tiempos estimados (funciona, pero sincroniza peor).

Recomendado

- **Avatares de Rin y Len** (y de nuevos personajes): renders con fondo claro uniforme, nombrados por reacción (`neutral.png` , `feliz_2.png` ...), y luego `npm run avatars:ingest`.
- **Más SFX con tags** (risa, idea, transición, error, éxito) de una biblioteca con licencia. El director los usará sin tocar código.
- **Ampliar el banco evergreen** a 50-80 temas ($\approx$ 4-6 meses a 3 videos por semana sin repetir). Los tests validan cada tema nuevo.
- **Plantillas de humor por personaje** (frases de firma, malentendidos típicos) para reforzar las personalidades.

Bloqueante para monetizar: licencias

Hoy el reporte marca `commercialUse: blocked`. Los avatares (renders MMD), la mayoría de los SFX, los memes y los GIFs de gatos tienen licencia *unknown*. Las ilustraciones oficiales de Miku, Rin, Len y Luka usan CC BY-NC (no comercial), y Teto tiene guías propias. Antes de activar Creator Rewards o YouTube Partner: reemplaza los SFX y memes por material licenciado, documenta el origen de cada avatar y revisa las guías de Crypton y de Teto para la monetización.

5. Presupuesto

Precios consultados en octubre de 2026 (verificar antes de contratar). Base: 3 videos por semana ($\approx$ 13 al mes), de unos 80 s y $\sim$ 1.300 caracteres de diálogo.

Concepto	Opción / precio de referencia	Costo / episodio	Costo / mes
Claude Code	Tu plan actual (desarrollo y mantenimiento)	—	ya cubierto

Escritor LLM (Claude API)	claude-opus-5-5: 4 USD/M tokens de entrada, 20 USD/M de salida. ~10 k de entrada y ~7 k de salida (con razonamiento) por intento; hasta 3 intentos	0,20-0,60 USD	3-8 USD
Voces (Fish Audio)	API ~15 USD por millón de caracteres, o plan Plus 15 USD/mes (~200 min). El plan gratuito (~7 min) da para pruebas	≈ 0,02 USD (API) 0,30 USD (API) · 15 USD (Plus)	
SFX y música con licencia	Epidemic Sound Creator: 9,99 USD/mes (anual), permite monetizar canales personales. Pro: 16,99 USD/mes si hay trabajos para clientes	—	10-17 USD
Remotion	Licencia gratuita para individuos y equipos de hasta 3 personas (revisa los términos vigentes)	—	0 USD
Whisper.cpp, FFmpeg, Node	Locales y gratuitos	—	0 USD
Render	Tu PC: ~5 min por video de 75 s con 4 núcleos (electricidad despreciable)	—	≈ 0 USD
Avatares nuevos	Renders MMD propios (solo tiempo) o ilustraciones encargadas (precio variable según artista)	—	único / variable

Escenario mínimo • ≈ 0-15 USD/mes

Solo temas evergreen (sin LLM), voces por API o con el plan gratuito/Plus, y SFX existentes. Sirve para validar el formato; **no monetizable** mientras haya licencias *unknown*.

Escenario recomendado • ≈ 30-45 USD/mes

Noticias con Claude, Fish Audio Plus y Epidemic Sound Creator. Material con licencia, listo para monetizar tras revisar los avatares.

A modo de referencia: Creator Rewards de TikTok exige 10.000 seguidores, 100.000 visualizaciones en 30 días y videos de más de 60 s. El escenario recomendado se paga con pocos videos que funcionen.

6. Primera corrida en tu máquina (checklist)

1. `git pull` y `npm install`. Python con Pillow, numpy y scipy si vas a ingerir avatares.
2. `npm run doctor` y `npm test` (deben dar 190 en verde).
3. `npm run autopilot -- --check-feeds`: corrige las fuentes que fallen.
4. `npm run autopilot -- --mode evergreen --tts files` (o `--tts fish`): revisa `projects/ep_.../script.md` y `autopilot.json` (avisos, SFX elegidos, notas).
5. Si el guion te convence: `npm run autopilot -- --episode <id> --produce`. Si no, edítalo a mano y produce.
6. Con `ANTHROPIC_API_KEY`: `npm run autopilot -- --mode news`. Verifica cada línea de `factClaims` contra las fuentes.
7. Publica con `output/<id>/publish/` (textos, horario y checklist). Marca el episodio como publicado en el historial (por ahora a mano).

7. Consejos

Calidad y confianza

- **Nunca publiques sin revisar.** En noticias, un error factual cuesta más que un video menos. El escritor LLM solo puede usar los hechos del brief, pero aun así puede equivocarse.
- **Etiqueta siempre el contenido de IA** en las tres plataformas (las voces imitan a personajes con voces humanas detrás). Es barato y la comunidad Vocaloid lo valora.
- **Varía de verdad.** YouTube penaliza el contenido masivo y con plantilla ("contenido no auténtico"). Alterna formatos, edita remates a mano y añade de vez en cuando episodios especiales hechos a mano.
- Evita chistes sobre nacionalidades y política. El prompt del guionista lo prohíbe y la regla de SFX para confusión excluye los memes con tag "china". Revisa también los SFX marcados a mano.

Producción

- **Lotes semanales:** un día, `--batch 3` + revisión (≈ 45 min); otro día, producción. Las noticias "calientes" van aparte y se publican de inmediato.
- **Haz crecer el banco evergreen** con temas que pida tu audiencia en los comentarios: es contenido gratuito, determinista y siempre disponible.
- Cuida el **historial** (`history.json`) en git: es lo que evita repetir temas y parejas.
- Mide y ajusta: retención, envíos por alcance y "visto frente a deslizado" (`docs/10`). Cambia **una sola** variable a la vez (formato, pareja, horario).

Técnicos

- No automatices capturas de artículos: riesgo de copyright, muros de pago y banners. Las tarjetas de titular propias son seguras.
- Remotion está fijado en 4.0.530 a propósito (4.0.531 rompe Studio). Comprueba Studio antes de actualizar.
- Si ejecutas como root (Docker/CI), Chrome necesita `--no-sandbox`; ya se aplica automáticamente en la generación de gráficos.

8. Próximos pasos sugeridos (priorizados)

#	Paso	Esfuerzo
1	Primera corrida local completa (sección 6) y ajuste de feeds y prompts	1-2 h
2	Resolver licencias: SFX y memes licenciados, origen de los avatares documentado	2-4 h + suscripción
3	Avatares y voz de Rin y Len (ingesta + <code>referenceId</code> + saludo)	2-3 h
4	Ampliar el banco evergreen a 50+ temas	continuo
5	Comando para marcar episodios publicados y registrar métricas (retención, envíos)	2 h (agente)
6	Generación de hashtags por plataforma con el LLM (hoy, reglas + entidades)	1 h (agente)
7	Más reglas de SFX y memes por personaje (firmas de Miku y Luka)	1 h + assets
8	Opcional: programación semanal con un cron local que ejecute <code>--batch 3</code> y avise para revisar	1 h

Anexo • Archivos principales

Ruta	Contenido
<code>scripts/autopilot.ts</code>	CLI del autopiloto
<code>scripts/ingest-avatars.ts</code>	CLI de ingesta de avatares
<code>src/autopilot/</code>	news, planner, template-writer, llm-writer, sfx-director, lint, graphics, backgrounds, publish, avatars, history, run
<code>config/autopilot/</code>	sources, evergreen (29 temas), formats, casting, themes, sfx-rules, humor
<code>prompts/writer.system.md</code>	Prompt del guionista LLM
<code>schemas/evergreen.schema.json</code>	Contrato del banco de temas
<code>tests/autopilot-*.test.ts</code>	Noticias, planificador, escritores, SFX, publicación, gráficos, avatares
<code>assets/backgrounds/bg_theme_*.mp4</code>	Fondos de los temas sakura, matrix, sunset y ocean
<code>docs/11_AUTOPILOT.md</code> , <code>docs/adr/0007-autopiloto.md</code>	Guía técnica y decisión de arquitectura

Fuentes de precios: Fish Audio (planes y API, reseñas de 2026), Epidemic Sound (planes de 2026), precios de la Claude API (oct. 2026). Licencias: Crypton/Piapro "For Creators" y guías de Kasane Teto (ver `docs/09_LICENSING.md`).